

Universidad de Buenos Aires
Facultad de Ciencias Exactas y Naturales
Departamento de Computación

Liquid Types para la verificación de *smart contracts* con diccionarios

Tesis de Licenciatura en Ciencias de la Computación

Tomás Jarabroviski

Director: Dr. Hernán Melgratti

Buenos Aires, 2026

Índice general

1.. Soporte de mappings 1
1.1. Semántica de un mapping 2
1.2. Especificación en Liquid Haskell 2
1.3. Traducción de las operaciones 4

1. SOPORTE DE MAPPINGS

Los *mappings* (diccionarios) son una de las estructuras de datos centrales en los contratos que manejan fondos: los saldos de cada cuenta en un token o los montos depositados por cada usuario en un banco se guardan en un *mapping* de direcciones en enteros, `mapping (address => uint)`. En [Con25], donde se presentó la traducción de Solidity a Haskell que este trabajo extiende, la posibilidad de representar *mappings* se estudió solo de forma preliminar, mediante una exploración manual que no llegó a incorporarse a la traducción automática. En este capítulo desarrollamos ese soporte de manera completa.

El problema tiene dos niveles. El primero es de especificación: definir en Haskell una representación de los *mappings* junto con tipos refinados que describan con precisión el efecto de sus operaciones, es decir, qué valor se lee después de una escritura, qué entradas no cambian y cómo evoluciona la suma de los valores. El segundo es de traducción: hacer que el traductor exprese cada operación de Solidity sobre un *mapping* en términos de esa representación, de modo que Liquid Haskell pueda verificar automáticamente las funciones del contrato que la usan. Los tipos refinados resultan adecuados para ambos niveles, porque permiten expresar los invariantes de la estructura y las garantías de cada operación como parte de los tipos, y delegar su verificación a un *solver* SMT [VSJ+14b].

A lo largo del capítulo utilizaremos como ejemplo el token mínimo del Listado 1.1. El constructor asigna todo el suministro inicial a la cuenta que despliega el contrato; `transfer` mueve `amount` unidades desde el saldo de quien invoca la función (`msg.sender`) hacia el de `to`, y `burn` destruye `amount` unidades del saldo de quien la invoca. Ambas funciones exigen, mediante `require`, que el saldo del remitente sea suficiente.

```
contract Token {
    mapping(address => uint) balances;

    constructor(uint initialSupply) {
        balances[msg.sender] = initialSupply;
    }

    function transfer(address to, uint amount) public {
        require(balances[msg.sender] >= amount);
        balances[msg.sender] -= amount;
        balances[to] += amount;
    }

    function burn(uint amount) public {
        require(balances[msg.sender] >= amount);
        balances[msg.sender] -= amount;
    }
}
```

Listado 1.1: Token mínimo con un *mapping* de saldos.

1.1. Semántica de un mapping

Un *mapping* de Solidity se comporta como una función total de claves en valores. La traducción debe preservar las siguientes propiedades:

Valor por defecto. Toda clave tiene un valor asociado: si nunca fue escrita, su valor es el valor por defecto del tipo, que para `uint` es 0. En el ejemplo, `balances[to]` vale 0 para cualquier cuenta que nunca recibió tokens. En consecuencia, un contrato no puede distinguir una clave que nunca fue escrita de una cuyo valor almacenado es 0, y `delete m[k]` equivale a `m[k] = 0`.

Escrituras locales. Las sentencias `m[k] = e`, `m[k] += e` y `m[k] -= e` modifican únicamente la entrada `k`; el resto del *mapping* no cambia.

Rango de los valores. Los valores de tipo `uint` son enteros entre 0 y $2^{256}-1$. La herramienta representa este rango con el predicado *InRange*, que tiene dos definiciones según el modo en que se ejecute:

- en el modo por defecto, $\mathit{InRange}(v) \equiv v \geq 0$, es decir, los enteros se interpretan como enteros matemáticos, sin cota superior;
- en el modo de detección de desbordamiento, que se presenta en un capítulo posterior, $\mathit{InRange}(v) \equiv 0 \leq v \leq 2^{256} - 1$.

En este capítulo usamos el modo por defecto, de modo que la única restricción sobre un valor almacenado es que sea mayor o igual a 0.

La última propiedad es la que convierte a los *mappings* en una fuente de errores. En `burn`, el `require` garantiza que el saldo resultante no sea negativo; si se omitiera, cualquier cuenta podría destruir tokens que no tiene. Queremos que la traducción transforme esta propiedad en una obligación de prueba, de modo que Liquid Haskell rechace la versión sin `require`.

Una alternativa natural sería representar el *mapping* como una función de Haskell `k -> Int`. Sin embargo, además de leer y escribir claves individuales, nos interesa poder razonar sobre la suma de todos los valores (por ejemplo, el total de tokens en circulación), lo cual requiere una representación finita y explícita de las entradas.

1.2. Especificación en Liquid Haskell

Las definiciones del Listado 1.2 forman parte de una biblioteca que se incluye en cada módulo generado que utiliza algún *mapping*.

```
{-@ type Map k v = {m : [(k, v)] | isUniq m} @-}

{-@ reflect sumBids @-}
sumBids :: Map k Int -> Int
sumBids []            = 0
sumBids ((_, x) : xs) = x + sumBids xs

{-@ data MapWithBalance k = MB { mMap :: Map k {v:Int | InRange v}
                               , mBal :: {v:Int | v >= 0} } @-}
```

```
{-@ type BalancedMap k =
      {m : MapWithBalance k | sumBids (mMap m) == mBal m} @-}
```

Listado 1.2: Representación de un *mapping* (definiciones principales).

Las definiciones se leen de la siguiente manera:

- `Map k v` es una lista de pares clave–valor. El predicado `isUniq` exige que ninguna clave aparezca dos veces, de modo que cada clave tiene a lo sumo un valor asociado.
- `sumBids` calcula la suma de los valores del mapa. La anotación `reflect` permite usar esta función dentro de los tipos refinados.
- `MapWithBalance k` acompaña al mapa con un campo `mBal` que guarda la suma de sus valores. Los valores del mapa deben cumplir el predicado *InRange* definido en la Sección 1.1.
- `BalancedMap k` es el tipo que efectivamente usa la traducción: un `MapWithBalance` cuyo campo `mBal` coincide con la suma real de sus valores. Este invariante es parte del tipo, por lo que toda función que devuelva un `BalancedMap` debe preservarlo, y Liquid Haskell lo verifica.

La lista de pares no aparece en el código generado para las funciones del contrato: este solo accede al mapa a través de las tres operaciones de la Tabla 1.1.

```
empWB          :: {m:BalancedMap k | mBal m == 0}
currentBalance :: m:BalancedMap k -> key:k -> {v:Int | InRange v}
setWB          :: m:BalancedMap k -> key:k -> val:{Int | InRange val}
               -> {n:BalancedMap k |
                     currentBalance n key == val &&
                     mBal n == mBal m - currentBalance m key + val}
```

Tab. 1.1: Interfaz de `BalancedMap` (tipos simplificados).

- `empWB` es el mapa vacío, cuya suma es 0.
- `currentBalance m key` devuelve el valor asociado a `key`, o 0 si la clave no está en el mapa. Es la operación que implementa la semántica de valor por defecto: internamente se distingue una clave ausente de una con valor 0, pero `currentBalance` devuelve lo mismo en ambos casos, tal como ocurre en Solidity.
- `setWB m key val` asocia `val` a `key`, ya sea agregando la clave o reemplazando su valor anterior, y actualiza la suma: se resta el valor previo y se suma el nuevo. Su argumento `val` debe cumplir *InRange*; esta precondición es la que permite detectar saldos negativos.

Los tipos de la Tabla 1.1 resumen las especificaciones de la biblioteca, que están expresadas en términos de la pertenencia de la clave al mapa y de la función `get`. Para enunciar con precisión las garantías que se obtienen de ellas usamos la siguiente notación: $m[k]$ denota `currentBalance m k`, $m[k \mapsto v]$ denota `setWB m k v`, y $\mathrm{dom}(m)$ es el conjunto

de claves presentes en m. Entonces, para todo mapa, todo par de claves y todo valor en rango,

$$\forall\, m : \mathit{BalancedMap}\ K, \quad \forall\, k, k' \in K, \quad \forall\, v \text{ tal que } \mathit{InRange}(v),$$

se cumplen las siguientes propiedades:

Lectura por defecto. $k \notin \mathrm{dom}(m) \;\Rightarrow\; m[k] = 0$.
Una clave ausente se lee como 0; en particular, toda lectura sobre `empWB` da 0.

Lectura tras escritura. $m[k \mapsto v][k] = v$.
Después de escribir v en la clave k, leer k devuelve v.

Marco. $k' \neq k \;\Rightarrow\; m[k \mapsto v][k'] = m[k']$.
Escribir una clave no modifica el valor de las demás.

Suma. $\mathit{mBal}(m[k \mapsto v]) = \mathit{mBal}(m) - m[k] + v$.
La suma se actualiza restando el valor anterior de la clave y sumando el nuevo.

Invariante. $\mathit{mBal}(m) = \sum_{(j,\,x) \in \mathit{mMap}(m)} x$.
El campo `mBal` coincide siempre con la suma de los valores del mapa; es el refinamiento que define al tipo `BalancedMap`.

Cota. $m[k] \leq \mathit{mBal}(m)$.
Ningún valor supera la suma, ya que todos los valores son no negativos.

Estas garantías se establecen dentro de la propia biblioteca, mediante lemas auxiliares sobre listas y conjuntos que no es necesario conocer para seguir la traducción.

1.3. Traducción de las operaciones

Recordemos brevemente el esquema de la traducción base [Con25]. El estado del contrato se representa con un registro de Haskell, y cada función de Solidity se traduce en una función que recibe el estado actual (`st`) y sus parámetros y devuelve el nuevo estado. La función `revert` tiene tipo `{v:String | False} -> a`, por lo que Liquid Haskell solo acepta un `revert` si puede probar que es inalcanzable. Sobre ese esquema:

- **Secuencia.** Una secuencia de sentencias se traduce como una cadena de estados intermedios `st1`, `st2`, ... ligados con `let`; el último de ellos es el resultado de la función.
- **`require`.** La sentencia `require(c)` se traduce de dos formas: como precondición en el tipo refinado de la función y como la sentencia `if c then st else revert "error"`, que continúa si la condición se cumple y revierte si no. La precondición es la que permite probar que ese `revert` es inalcanzable.
- **`assert`.** La sentencia `assert(c)` se traduce como `if c then st else revert "assert "`, en el lugar donde está escrita. Como en este caso no hay una precondición que lo respalde, Liquid Haskell debe probar que `c` se cumple siempre en ese punto de la función. Además, los `assert` con los que termina la función se agregan como postcondiciones de su tipo refinado. Esto modifica la traducción de [Con25], donde todo `assert` se traducía únicamente como postcondición, de modo que un `assert` escrito en medio de la función se verificaba sobre el estado final y no sobre el estado en el que está escrito.

Con estas reglas como base, la Tabla 1.2 muestra cómo se traducen las operaciones sobre un *mapping* `m` declarado en un contrato `C`. Al igual que el resto de los campos del estado, su nombre lleva como sufijo el nombre del contrato.

Solidity	**Haskell**
`mapping(K => uint) m`	campo `mC :: BalancedMap K`
valor inicial	`empWB`
`m[k]`	`currentBalance (mC st) k`
`m[k] = e`	`setWB (mC st) k e`
`m[k] += e`	`setWB (mC st) k (currentBalance (mC st) k + e)`
`m[k] -= e`	`setWB (mC st) k (currentBalance (mC st) k - e)`

Tab. 1.2: Traducción de las operaciones sobre un *mapping*. En las tres escrituras, el mapa resultante se asigna al campo `mC` del nuevo estado.

A continuación aplicamos la traducción al token del Listado 1.1.

Estado. El estado del token se representa con el registro `TokenState` del Listado 1.3.

```
data TokenState = TokenState {
  msgToken      :: Message,
  blockToken    :: Block,
  balanceToken  :: Amount,
  addressToken  :: Address,
  balancesToken :: BalancedMap Address
}
```

Listado 1.3: Estado del token.

Los cuatro primeros campos se agregan en todo contrato: el mensaje de la invocación en curso (que contiene `msg.sender` y `msg.value`), los datos del bloque, la cantidad de ether que posee el contrato y su propia dirección. El último campo es el *mapping* `balances`.

Constructor. El constructor construye el estado inicial completo a partir del mensaje y el bloque del despliegue. El *mapping* parte del mapa vacío y, como el constructor escribe una clave, se le aplica la escritura correspondiente:

```
{-@ constructorToken :: Message -> Block
                     -> {initialSupply:Uint | True}
                     -> {v:TokenState | True} @-}
constructorToken msgctx block initialSupply = TokenState
  { msgToken      = msgctx
  , blockToken    = block
  , balanceToken  = 0
  , addressToken  = fromString "Token"
  , balancesToken = setWB empWB (sender msgctx) initialSupply }
```

A diferencia del resto de las funciones, el constructor no recibe un estado previo: recibe el mensaje (`msgctx`) y el bloque (`block`) del despliegue, y su tipo refinado no tiene precondiciones porque no contiene `require`. El contrato comienza sin ether, porque el constructor no es `payable`, y su dirección se obtiene a partir de su nombre. La obligación que impone

`setWB` es que `initialSupply` sea no negativo, lo cual se cumple porque el parámetro tiene tipo `Uint`.

`burn`. El Listado 1.4 muestra el código generado para `burn`. Para facilitar la lectura abreviamos `balancesToken` como `bal`, el remitente `sender (msgToken st)` como `me` (es el mismo en todos los estados intermedios) y el mapa actualizado `balancesTokenNext` como `next`; por lo demás, el código es el que produce la herramienta.

```
{-@ burn :: {st:TokenState | True}
         -> {amount:Uint | currentBalance (bal st) me >= amount}
         -> {v:TokenState | True} @-}
burn st amount = let
  st1 = if currentBalance (bal st) me >= amount
          then st else revert "error"
  st2 = let next = setWB (bal st1) me (currentBalance (bal st1) me - amount)
        in assert (lemma_currentBalance_get (bal st1) me) $
           assert (lemma_currentBalance_get next me) $
           st1 { bal = next }
  in st2
```

Listado 1.4: Traducción de `burn`.

El `require` aparece dos veces: como precondición y como la sentencia que define `st1`. La precondición menciona al parámetro `amount`, por lo que se escribe como refinamiento de ese parámetro y no del estado: en general, una condición que menciona parámetros se asocia al último de ellos en la firma, de modo que todos estén ligados al leerla. La postcondición es `True`, porque la función no contiene `assert`.

La escritura `balances[msg.sender] -= amount` se traduce en `st2`. Para aceptar la llamada a `setWB`, Liquid Haskell debe probar que el nuevo valor, `currentBalance (bal st1) me - amount`, es no negativo. Esto se deduce de la precondición, ya que `st1` es igual a `st`. Las dos aserciones que acompañan a la escritura se explican al final de la sección.

Si se elimina el `require` de `burn`, la precondición desaparece y esa obligación ya no puede probarse: Liquid Haskell rechaza la llamada a `setWB` y la herramienta reporta el contrato como inseguro. El error de programación (permitir que una cuenta destruya tokens que no tiene) se manifiesta como un error de tipos en el código traducido.

`transfer`. Con las mismas abreviaturas, el tipo refinado de `transfer` es

```
{-@ transfer :: {st:TokenState | True}
             -> {to:Address | True}
             -> {amount:Uint | currentBalance (bal st) me >= amount}
             -> {v:TokenState | True} @-}
```

y tiene la misma forma que el de `burn`: la condición del `require` queda asociada a `amount`, el único parámetro que menciona, y la postcondición es `True` porque la función tampoco contiene `assert`. Las dos escrituras se traducen en dos estados consecutivos (omitimos las aserciones de lemas, que son las mismas que en `burn`):

```
  st2 = st1 { bal = setWB (bal st1) me (currentBalance (bal st1) me - amount) }
  st3 = st2 { bal = setWB (bal st2) to (currentBalance (bal st2) to + amount) }
```

La primera escritura genera la misma obligación que en `burn`, que se cumple gracias al `require`. La segunda obligación es inmediata: la suma de dos valores no negativos es no negativa. Notemos que la segunda escritura lee el saldo de `to` a partir de `st2`, es decir, después de la primera escritura. De esta forma la traducción es correcta incluso cuando `to` y `msg.sender` son la misma cuenta, sin necesidad de tratar ese caso por separado.

Supongamos ahora que el programador se equivoca de clave y escribe `require(balances [to] >= amount)`. La precondición pasa a hablar del saldo de `to`, que no dice nada sobre el saldo de `msg.sender`; la obligación de la primera escritura deja de poder probarse, y la herramienta reporta nuevamente el contrato como inseguro. En cambio, la versión original del Listado 1.1 es verificada por Liquid Haskell.

Lemas en el código generado. En el Listado 1.4, la escritura está acompañada de dos aserciones de la forma `assert (lemma ...) $ ...`. La función `assert :: a -> b -> b` ignora su primer argumento y devuelve el segundo; su único propósito es incorporar al contexto de Liquid Haskell el enunciado del lema. En este caso el lema es `lemma_currentBalance_get m k`, que relaciona `currentBalance` con la función `get`: si $k \in \mathrm{dom}(m)$, entonces $m[k]$ es el valor que devuelve `get`, y si no, $m[k] = 0$. Como las especificaciones de `setWB` están expresadas en términos de `get`, este lema es el que permite trasladarlas a las lecturas con `currentBalance` que aparecen en el código del contrato.

El traductor inserta los lemas según una regla fija. En cada escritura `m[k] = e`, `m[k] += e` o `m[k] -= e`:

- siempre, `lemma_currentBalance_get` sobre el mapa anterior y sobre el mapa resultante, para la clave `k`;
- en el modo de detección de desbordamiento, además, para cada otra clave del mismo *mapping* que lea la función, el lema de marco `lemma_currentBalance_setWB_maybe_other`;
- si el *mapping* registra el ether recibido por el contrato, además, el lema de cota `lemma_currentBalance_le_mBal` para la clave escrita y para cada clave del mismo *mapping* que lea la función.

Las dos últimas situaciones se presentan en capítulos posteriores. En el token, que se verifica en el modo por defecto y no recibe ether, solo aparecen las dos aserciones del primer caso.

Bibliografía

[Con25] Gonzalo Consoli. Liquid types para la verificación de smart contracts. Tesis de licenciatura, Departamento de Computación, Facultad de Ciencias Exactas y Naturales, Universidad de Buenos Aires, septiembre 2025. Director: Dr. Hernán Claudio Melgratti.

[HVH19] Martin A. T. Handley, Niki Vazou, and Graham Hutton. Liquidate your assets: Reasoning about resource usage in Liquid Haskell. *Proceedings of the ACM on Programming Languages*, 4(POPL):1–27, 2019.

[LPR+20] Yiyun Liu, James Parker, Patrick Redmond, Lindsey Kuper, Michael Hicks, and Niki Vazou. Verifying replicated data types with typeclass refinements in Liquid Haskell. *Proceedings of the ACM on Programming Languages*, 4(OOPSLA):1–30, 2020.

[NSC+22] Yuki Nishida, Hiromasa Saito, Ran Chen, Akira Kawata, Jun Furuse, Kohei Suenaga, and Atsushi Igarashi. Helmholtz: A verifier for Tezos smart contracts based on refinement types. *New Generation Computing*, 40(2):507–540, 2022.

[RKJ08] Patrick M. Rondon, Ming Kawaguci, and Ranjit Jhala. Liquid types. In *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 159–169, 2008.

[Sza97] Nick Szabo. Formalizing and securing relationships on public networks. *First Monday*, 1997.

[VSJ14a] Niki Vazou, Eric L. Seidel, and Ranjit Jhala. LiquidHaskell: Experience with refinement types in the real world. In *Proceedings of the 2014 ACM SIGPLAN Symposium on Haskell*, pages 39–51, 2014.

[VSJ+14b] Niki Vazou, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton-Jones. Refinement types for Haskell. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming*, pages 269–282, 2014.